

Design and Analysis of Algorithms

- *Course Code:* BCSE204L
- *Course Type:* Theory (ETH)
- *Slot:* A1+TA1 & & A2+TA2
- *Class ID:* VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 3

Module:3	String Matching Algorithms
----------	----------------------------

5 hours

Naïve String-matching Algorithms,
Knuth-Morris-Pratt (KMP) algorithm,
Rabin-Karp Algorithm,
Suffix Trees.

Rabin-Karp Algorithm (Introduction)

- The Rabin-Karp Algorithm is a string-searching algorithm that efficiently finds the occurrence of a pattern within a text using hashing.
- The algorithm compares the **hash value** of the pattern with the hash values of substrings of the text.
- If the hash values match, it compares the pattern and the substring character by character to confirm the match.
- It's particularly useful when you need to find multiple occurrences of a pattern in a text.

How the Rabin-Karp Algorithm works

- S1: Calculate the hash value of the pattern.
- S2: Calculate the hash value of the first window of the text with the same length as the pattern.
- S3: Compare the hash values:
 - If they match, compare the pattern and the text window character by character to confirm the match.
 - If they don't match, move the window by one position and recalculate its hash value.
- S4: Repeat step 3 until a match is found or until the end of the text is reached.

- Given Text = 315265 and Pattern = 26
- We choose $b = 11$
- $P \bmod b = 26 \bmod 11 = 4$

Pattern Hash value=4

3 1 5 2 6 5 $31 \bmod 11 = 9$ not equal to 4

3 1 5 2 6 5 $15 \bmod 11 = 4$ equal to 4 -> spurious hit

3 1 5 2 6 5 $52 \bmod 11 = 8$ not equal to 4

3 1 5 2 6 5 $26 \bmod 11 = 4$ equal to 4 -> an exact match!!

3 1 5 2 6 5 $65 \bmod 11 = 10$ not equal to 4

As we can see, when a match is found, further testing is done to ensure that a match has indeed been found.

Text: ABCCDDAEFG

A	B	C	C	D	D	A	E	F	G
---	---	---	---	---	---	---	---	---	---

Pattern: CDD

C	D	D
---	---	---

Let us assign a numerical value(v)/weight for the characters we will be using in the problem. Here, we have taken first ten alphabets only (i.e. A to J).

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10

n be the length of the pattern and m be the length of the text. Here, $m = 10$ and $n = 3$.

Let d be the number of characters in the input set. Here, we have taken input set $\{A, B, C, \dots, J\}$.

So, $d = 10$. You can assume any suitable value for d .

Let us calculate the hash value of the pattern.

C	D	D
---	---	---

hash value = 6

$$\begin{aligned}\text{hash value for pattern}(p) &= \Sigma(v * d^{(m-i-1)}) \bmod 13, \text{ where } i=0 \text{ to } m \\ &= ((3 * 10^2) + (4 * 10^1) + (4 * 10^0)) \bmod 13 \\ &= 344 \bmod 13 \\ &= 6\end{aligned}$$

Text: ABCCDDAEFG

A B C C D D A E F G

Pattern: CDD

C D D

Hash value of the pattern.

C D D
hash value = 6

A B C D E F G H I J
1 2 3 4 5 6 7 8 9 10

Calculate the hash value for the text-window of size m.

For the first window **ABC**,

hash value for text(t) = $\sum(v * d^{(m-i-1)}) \bmod 13$, where $i=0$ to m

$$\begin{aligned} &= ((1 * 10^2) + (2 * 10^1) + (3 * 10^0)) \bmod 13 \\ &= 123 \bmod 13 \\ &= 6 \end{aligned}$$

A B C C D D A E F G
C D D

Compare the hash value of the pattern with the hash value of the text. If they match then, character-matching is performed.

In the above examples, the hash value of the first window (i.e. t) matches with p so, go for character matching between **ABC** and **CDD**.

Since they do not match so, go for the next window.

We calculate the hash value of the next window by subtracting the first term and adding the next term.

Text: ABCCDDAEFG

A	B	C	C	D	D	A	E	F	G
---	---	---	---	---	---	---	---	---	---

Pattern: CDD

C	D	D
---	---	---

Hash value of the pattern.

C	D	D
---	---	---

hash value = 6

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10

Calculate the hash value for the text-window of size m.

For the second window **BCC**,

hash value for text(t) = $\Sigma(v * d^{(m-i-1)}) \bmod 13$, where $i=0$ to m

$$= ((2 * 10^2) + (3 * 10^1) + (3 * 10^0)) \bmod 13$$

$$= 233 \bmod 13$$

$$= 12$$

A	B	C	C	D	D	A	E	F	G

Compare the hash value of the pattern with the hash value of the text.

For **BCC**, $t = 12 (\neq 6)$. Therefore, go for the next window.

We calculate the hash value of the next window by subtracting the first term and adding the next term.

Text: ABCCDDAEFG

A B C C D D A E F G

Pattern: CDD

C D D

Hash value of the pattern.

C D D
hash value = 6

Calculate the hash value for the text-window of size m.

For the second window **CCD**,

hash value for text(t) = $\sum(v * d^{(m-i-1)}) \bmod 13$, where $i=0$ to m

$$\begin{aligned} &= ((3 * 10^2) + (3 * 10^1) + (4 * 10^0)) \bmod 13 \\ &= 334 \bmod 13 \\ &= 9 \end{aligned}$$

A B C D E F G H I J
1 2 3 4 5 6 7 8 9 10

A B C C D D A E F G
[] [] []

Compare the hash value of the pattern with the hash value of the text.

For **CCD**, $t = 9 (\neq 6)$. Therefore, go for the next window.

We calculate the hash value of the next window by subtracting the first term and adding the next term.

Text: ABCCDDAEFG

A B C C D D A E F G

Pattern: CDD

C D D

Hash value of the pattern.

C D D
hash value = 6

Calculate the hash value for the text-window of size m.

For the second window **CDD**,

hash value for text(t) = $\sum(v * d^{(m-i-1)}) \bmod 13$,
where $i=0$ to m

$$\begin{aligned} &= ((3 * 10^2) + (4 * 10^1) + (4 * 10^0)) \bmod 13 \\ &= 344 \bmod 13 \\ &= 6 \end{aligned}$$

Compare the hash value of the pattern with the hash value of the text.

For CDD, $t = 6$ (=6).

They are matched, character-matching is performed.

In the above examples, the hash value of the first window (i.e. t) matches with p so, go for character matching between **CDD** and **CDD**. They are matched,

Continu this procedure for $N-M+1$ times to find

A B C D E F G H I J
1 2 3 4 5 6 7 8 9 10

A B C C D D A E F G
C D D

Same example with
Different Hash function.

Text: ABCCDDAEFG

Pattern: CDD

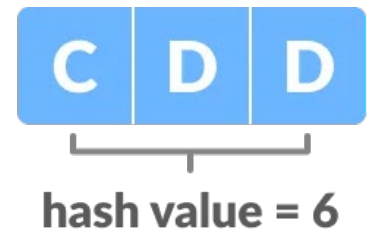


let's recalculate the hash values using ASCII values instead of custom numerical values.
In ASCII encoding, the characters 'A' to 'Z' have ASCII values from 65 to 90, and 'a' to 'z' have ASCII values from 97 to 122.

n be the length of the pattern and **m** be the length of the text. Here, $m = 10$ and $n = 3$.
We'll use the ASCII values of characters and calculate the hash values accordingly.

Calculate the hash value of the pattern (**CDD**)

$$\begin{aligned}\text{hash value for pattern}(p) &= \sum_{i=0}^{n-1} (\text{ASCII value of pattern}[i]) \times d^{(m-i-1)} \mod 13 \\ &= (67 \times 10^2 + 68 \times 10^1 + 68 \times 10^0) \mod 13 \\ &= (6700 + 680 + 68) \mod 13 \\ &= 7448 \mod 13 \\ &= 6\end{aligned}$$



Text: ABCCDDAEFG

A B C C D D A E F G

Hash value of the pattern.

C D D
hash value = 6

Pattern: CDD

C D D

Calculate the hash value for each text window of size m using ASCII values:

For the first window "ABC":

$$\text{hash value for text}(t) = \sum_{i=0}^{n-1} (\text{ASCII value of text}[i]) \times d^{(m-i-1)} \mod 13$$

$$= (65 \times 10^2 + 66 \times 10^1 + 67 \times 10^0) \mod 13$$

$$= (6500 + 660 + 67) \mod 13$$

$$= 717 \mod 13$$

$$= 12$$

A B C C D D A E F G

Compare the hash value of the pattern with the hash value of the text.

For **ABC**, $t = 12 (\neq 6)$. Therefore, go for the next window.

Note: ASCII encoding, the characters 'A' to 'Z' have ASCII values from 65 to 90

Text: ABCCDDAEFG

A B C C D D A E F G

Hash value of the pattern.

C D D
hash value = 6

Pattern: CDD

C D D

Calculate the hash value for each text window of size m using ASCII values:

For the first window “**BCC**”:

$$\begin{aligned}\text{hash value for text}(t) &= \sum_{i=0}^{n-1} (\text{ASCII value of text}[i]) \times d^{(m-i-1)} \mod 13 \\ &= (66 \times 10^2 + 67 \times 10^1 + 67 \times 10^0) \mod 13 \\ &= (6600 + 670 + 67) \mod 13 \\ &= 727 \mod 13 \\ &= 10\end{aligned}$$

A B C C D D A E F G
 □ □ □

Compare the hash value of the pattern with the hash value of the text.

For **BCC**, $t = 10$ ($\neq 6$). Therefore, go for the next window.

Note: ASCII encoding, the characters 'A' to 'Z' have ASCII values from 65 to 90

Text: ABCCDDAEFG

A B C C D D A E F G

Hash value of the pattern.

C D D
hash value = 6

Pattern: CDD

C D D

Calculate the hash value for each text window of size m using ASCII values:

For the first window “**CCD**”:

A B C C D D A E F G
 □ □ □

$$\text{hash value for text}(t) = \sum_{i=0}^{n-1} (\text{ASCII value of text}[i]) \times d^{(m-i-1)} \mod 13$$

$$= (67 \times 10^2 + 67 \times 10^1 + 68 \times 10^0) \mod 13$$

$$= (6700 + 670 + 68) \mod 13$$

$$= 737 \mod 13$$

$$= 11$$

Compare the hash value of the pattern with the hash value of the text.

For **CCD**, $t = 11 (\neq 6)$. Therefore, go for the next window.

Text: ABCCDDAEFG

A B C C D D A E F G

Hash value of the pattern.

C D D
hash value = 6

Pattern: CDD

C D D

Calculate the hash value for each text window of size m using ASCII values:

For the first window “CDD”:

$$\begin{aligned}\text{hash value for text}(t) &= \sum_{i=0}^{n-1} (\text{ASCII value of text}[i]) \times d^{(m-i-1)} \mod 13 \\ &= (67 \times 10^2 + 68 \times 10^1 + 68 \times 10^0) \mod 13 \\ &= (6700 + 680 + 68) \mod 13 \\ &= 7448 \mod 13 \\ &= 6\end{aligned}$$

A B C C D D A E F G

C D D

Compare the hash value of the pattern with the hash value of the text.

For **CDD**, $t = 6$ ($=6$).

They are matched, character-matching is performed.

In the above examples, the hash value of the first window (i.e. t) matches with p so, go for character matching between **CDD** and **CDD**. They are matched,

Time Complexity Analysis

- Calculating the hash value of the pattern and the first window of the text takes $O(m)$ time.
- Comparing hash values and comparing substrings in each iteration takes $O(1)$ time on average.
- The worst-case time complexity is $O((n - m + 1) * m)$ where n is the length of the text and m is the length of the pattern.
- However, with a good hash function and appropriate choice of hash base, Rabin-Karp can achieve linear time complexity $O(n + m)$ on average.

Key Points:

- Rabin-Karp Algorithm leverages hashing to efficiently search for a pattern within a text.
- It can handle multiple occurrences of the pattern within the text.
- The choice of hash function and base affects the efficiency of the algorithm.
- It's particularly useful when the pattern and text are large or when there are multiple patterns to search for within the text.

Any

Question



PresenterMedia



Thank You!

**FOR YOUR
ATTENTION**

